

# Secure Identity Store Design

## Secure Identity Store — Design (identity-share-discovery epic)

**Revision:** 1 **Last modified:** 2026-06-26T11:21:58Z **Scope:** Encrypted-at-rest storage of network-share identities (credentials = username+password, api\_token, ssh\_key) for the catalog-api scanner, replacing the plaintext StorageRoot.Password debt. **Authority:** §11.4.10 (no plaintext secret leak), §11.4.74 (catalogue-first reuse), §11.4.6 (no guessing — every claim cites a file:line read), §9.2 (data safety / rollback).

---

### 0. The §11.4.10 debt (FACT)

Plaintext credentials are stored today:

- catalog-api/models/file.go:56 — Password \*string ... db:"password" on StorageRoot.
- catalog-api/models/file.go:55 — Username \*string ... db:"username".
- catalog-api/database/migrations/000001\_initial\_schema.sqlite.up.sql:4 CREATE TABLE ... storage\_roots, line :11 username TEXT, line :12 password TEXT — the column is plaintext TEXT.
- The value reaches config from the environment in plaintext: catalog-api/config/config.go:238–239 (DATABASE\_PASSWORD → config.Database.Password) and the .env.example SMB\_PASSWORD=password placeholder.
- StorageRoot is consumed by the scanner/handlers: catalog-api/handlers/admin\_handler.go, catalog-api/handlers/browse.go, catalog-api/main.go (verified via grep of non-test StorageRoot references).

A password TEXT column means anyone with read access to the SQLite/Postgres file or a DB backup reads every share password. This is a §11.4.10 violation that the epic must close.

---

## 1. Submodule reuse survey (§11.4.74)

The catalogue-first survey of submodules/ for an existing secure-storage / secrets / keyring / encryption capability.

### **Candidate A — `digital.vasic.security/pkg/securestorage` — VERDICT: REUSE (primary, ≥80% fit)**

- Path: `submodules/security/pkg/securestorage/securestorage.go`.
- Module: `digital.vasic.security` (`submodules/security/go.mod:1`), depends on `golang.org/x/crypto v0.52.0` (`submodules/security/go.mod:9`) but the secure-storage primitive itself uses only `stdlib crypto/aes + crypto/cipher` (`securestorage.go:4–5`).
- **Already wired into catalog-api:** `catalog-api/go.mod:43` replace `digital.vasic.security` => `../submodules/security` and `catalog-api/go.mod:72` require `digital.vasic.security`. No new dependency wiring needed — this is the strongest reuse signal.
- Exported API (verified by reading the file):
  - `type Storage interface { Store, Retrieve, Delete, Contains, ListKeys, Clear, IsSecure } — securestorage.go:18–26.`
  - `func NewFileStorage(storageDir string) *FileStorage — securestorage.go:41.`
  - Scheme-aware convenience methods that map 1:1 to the three identity schemes the epic needs:
    - `func (fs *FileStorage) StoreCredentials(service, username, password string) error — securestorage.go:175; retrieve at securestorage.go:181 (RetrieveCredentials).` Length-prefixed encoding (`securestorage.go:176`) so username/password round-trip exactly.
    - `func (fs *FileStorage) StoreToken(service, token string) error — securestorage.go:206; RetrieveToken securestorage.go:211 → covers api_token.`
    - `func (fs *FileStorage) StorePrivateKey(service, privateKey string) error — securestorage.go:216; RetrievePrivateKey securestorage.go:221 → covers ssh_key.`
- Crypto primitive (verified): AES-256-GCM. `encrypt()` `securestorage.go:332–347` — `aes.NewCipher + cipher.NewGCM`, random 12-byte nonce via `crypto/rand` (`securestorage.go:341–342`), nonce prepended to ciphertext, base64 output (`securestorage.go:345–346`).

- decrypt() securestorage.go:349–372 with a ciphertext too short guard (securestorage.go:363–364). 32-byte key ⇒ AES-256.
- Concurrency: sync.RWMutex, and the read paths deliberately take the write lock because getOrCreateKey/loadCache mutate shared state — documented at securestorage.go:74–78, :110, :123, :150. This is production-grade, not a stub (§11.4.27 — no fakes outside unit tests).
- Hardening already present: input-validation against panics in RetrieveCredentials (negative length-prefix guard, securestorage.go:194–201); key file 0600, dir 0700 (securestorage.go:226, :248); test suite is extensive (securestorage\_race\_test.go, \_gap\_test.go, \_neglen\_test.go, \_coverage\_test.go).

**Why ≥80%, not 100%:** securestorage persists to its OWN files (.storage\_key + .secure\_storage, securestorage.go:44–45, persist() :310), not to the catalog-api SQLite/Postgres DB, and its AEAD encrypt/decrypt are **unexported** (securestorage.go:332, :349). The master key is a sibling file written 0600 (getOrCreateKey securestorage.go:229–253) — there is no env/OS-keyring/KMS key source. The two gaps (DB-backed ciphertext column; master-key provenance) are what we **extend upstream** (§11.4.74), not reimplement.

### **Candidate B — digital.vasic.security/pkg/e2ee — VERDICT: REUSE for the AEAD primitive if a DB-column codec is preferred**

- Path: submodules/security/pkg/e2ee/ (chacha.go, sse.go, package.go, transport.go).
- Exported (verified): type AEADSuite uint8 (chacha.go:26); StreamSealer/StreamDecryptor with NewStreamSealer(sess, aad) (sse.go:153–161) and NewStreamDecryptor(sess, aad) (sse.go:45–60). ChaCha20-Poly1305 (RFC 8439) is operator-authorized for this module per submodules/security/CLAUDE.md (“Dependencies” note).
- Fit: e2ee is built for **session/transport** sealing (a Session, AAD, streaming), not at-rest key/value secrets. It is heavier than the epic needs. Use it only if we want a single project-wide AEAD codec; otherwise Candidate A is the simpler at-rest fit.

### **Candidate C — digital.vasic.storage/pkg/s3 (config) — VERDICT: NO-MATCH (it is a consumer of secrets, not a store)**

- Path: submodules/storage/pkg/s3/config.go / credentials handling. This module *consumes* access/secret keys to talk to S3; it does not

provide encrypted-at-rest local secret storage. Not the capability we need.

### **Candidate D — `digital.vasic.auth/pkg/*` — VERDICT: NO-MATCH (token issuance, not secret-at-rest)**

- Path: `submodules/auth/pkg/{jwt,token,tokenmanager,apikey,accesstoken}/*.go` (module `digital.vasic.auth, submodules/auth/go.mod:1`). These issue/verify JWT/API keys; they are not an encrypted secret vault for third-party share credentials. Not a fit.

### **Candidate E — OS keyring submodule — VERDICT: NO-MATCH found**

- No keyring/keychain/secret-service submodule exists under `submodules/` (directory listing inspected; only `security, storage, auth, memory, helix_memory` carry crypto-adjacent code). An OS-keyring backend, if desired, is an **upstream extension** of `digital.vasic.security` (§11.4.74), not an in-catalog-api reimplementation.

## **Reuse decision**

Build the secure identity store on **`digital.vasic.security/pkg/securestorage`** (Candidate A). Extend that package **upstream** with exactly two additions so it covers 100% of the epic:

1. **Exported AEAD codec** — promote `encrypt/decrypt` (`securestorage.go:332,:349`) to an exported func `Seal(plaintext string, key []byte) (string, error) / func Open(ciphertext string, key []byte) (string, error)`, or add a type `Cipher` so a caller can encrypt a value destined for a DB column without the file-backed `FileStorage`. This lets `catalog-api` keep its single source of truth (the DB) while reusing the audited primitive.
2. **Pluggable master-key source** — add `func NewFileStorageWithKey(dir string, key []byte) *FileStorage` (or a `KeyProvider` interface) so the 32-byte key can come from `env / OS keyring / KMS` instead of the `.storage_key` sibling file (`securestorage.go:44`). The default file behaviour stays for callers that want it.

Both are generic, project-agnostic additions (they name no consumer) — compliant with the security module's decoupling rules in `submodules/security/CLAUDE.md`.

---

## 2. Secure identity store — schema & model

### 2.1 New table `share_identity`

A single multi-scheme identity row. The secret material is **never** stored plaintext; only an AEAD ciphertext blob is persisted.

```
-- migration 0NN_create_share_identity (SQLite shown; Postgres
variant mirrors it)
CREATE TABLE IF NOT EXISTS share_identity (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  name        TEXT NOT NULL,
  -- operator-facing label, NOT a secret
  scheme      TEXT NOT NULL,
  -- 'credentials' | 'api_token' | 'ssh_key'
  username    TEXT,                                -- non-secret
  -- principal (credentials/ssh user)
  domain      TEXT,                                -- SMB domain,
  -- non-secret
  secret_ct   TEXT NOT NULL,
  -- AEAD ciphertext (base64), NEVER plaintext
  key_id      TEXT NOT NULL,                        -- which master
  -- key/DEK encrypted secret_ct
  enc_alg     TEXT NOT NULL DEFAULT 'AES-256-GCM',
  created_at  TIMESTAMP NOT NULL,
  updated_at  TIMESTAMP NOT NULL
);
CREATE UNIQUE INDEX IF NOT EXISTS idx_share_identity_name ON
share_identity(name);
```

- scheme is a closed set matching the epic's three identity types.
- For scheme='credentials': plaintext username (non-secret) lives in its own column; the **password** is inside secret\_ct. (Mirrors securestorage.StoreCredentials length-prefixed split at securestorage.go:176, but we keep the username in the clear column for queryability and put only the password under AEAD.)
- For scheme='api\_token': the token is the whole secret\_ct; username NULL.
- For scheme='ssh\_key': username = ssh login (non-secret); the PEM private key is secret\_ct. (Maps to securestorage.StorePrivateKey, securestorage.go:216.)
- key\_id + enc\_alg make key rotation and algorithm migration auditable without re-reading every blob blindly.

### 2.2 `share_identity_binding` — reference by ID, never duplicate the secret

```
CREATE TABLE IF NOT EXISTS share_identity_binding (
  storage_root_id INTEGER NOT NULL REFERENCES
  storage_roots(id) ON DELETE CASCADE,
```

```

identity_id      INTEGER NOT NULL REFERENCES
share_identity(id) ON DELETE RESTRICT,
PRIMARY KEY (storage_root_id, identity_id)
);

```

- A StorageRoot (models/file.go:48–70) references an identity **by identity\_id** — the secret is stored exactly once in share\_identity and shared by reference. No password column on storage\_roots, no copy of the secret anywhere.
- ON DELETE RESTRICT on identity\_id prevents deleting an identity still in use; ON DELETE CASCADE on the root cleans the binding when a root is removed.
- The Go model gains IdentityID \*int64 \json:“identity\_id,omitempty” db:“identity\_id” onStorageRoot(inmodels/file.go), and thePassword \*string db:“password”field atmodels/file.go:56` is **removed** (see §3 migration).

## 2.3 Encryption approach — envelope encryption, master key NOT in the DB

Recommended: **envelope encryption** reusing the AES-256-GCM primitive that securestorage already ships (securestorage.go:332–372).

- A **master key (KEK)**, 32 bytes, sourced (in precedence order, mirroring catalog-api’s existing env-first config convention at config/config.go:221–298):
    1. HELIX\_IDENTITY\_MASTER\_KEY env var (base64 32 bytes) — authoritative, git-ignored per §11.4.10 / §11.4.30, documented as a placeholder in .env.example (which today only carries SMB\_PASSWORD=password — that placeholder is retired).
    2. OS keyring (upstream securestorage KeyProvider extension, §1).
    3. A 0600 key file via the existing securestorage file behaviour (securestorage.go:248) — dev fallback only.
  - Per-identity **data key (DEK)** OR direct KEK encryption: for the volume here (a handful of share identities) direct AES-256-GCM under the KEK is sufficient; key\_id records the KEK version so rotation re-encrypts blobs without ambiguity.
  - **Why this over alternatives:** the codebase already builds on stdlib AES-256-GCM (securestorage.go) and has no age/external-KMS dependency in any consumed submodule (verified — no filippo.io/age in submodules/security/go.mod). Choosing the primitive already wired in (go.mod:43/72) honours §11.4.74 (reuse over new dependency) and §11.4.8 (no novel crypto). External KMS (AWS/Vault) can later become an additional KeyProvider source without schema change.
-

### 3. Migration plan — remove plaintext `storage_roots.password`

catalog-api uses a versioned migration runner (database/migrations.go:16, dual SQLite/Postgres variants under database/migrations/). Add three forward steps:

1. **0NN\_add\_share\_identity\_up** — create `share_identity` + `share_identity_binding` (§2). No data change yet. Reversible down-migration drops both tables.
2. **0NN+1\_migrate\_passwords\_up** — a one-shot data migration (Go-side, like `migrateSMBToStorageRoots` at `database/migrations.go:16`):
  - For each `storage_roots` row with non-NULL password (column at `migrations/000001_initial_schema.sqlite.up.sql:12`): create a `share_identity` row (`scheme='credentials'`, `username` from `storage_roots.username:11`, `secret_ct = Seal(password, KEK)` using the extended exported codec from §1), insert a `share_identity_binding`, then **NULL out** `storage_roots.password`.
  - Idempotent: skip roots that already have a binding (re-runnable per §11.4.50).
  - Runs inside a transaction; on any row failure the whole step rolls back (the runner already wraps Up funcs).
3. **0NN+2\_drop\_password\_column\_up** — drop `storage_roots.password` (SQLite: table-rebuild without the column, the pattern this codebase already uses; Postgres: `ALTER TABLE storage_roots DROP COLUMN password`). Down-migration re-adds an empty password TEXT (data is NOT restorable plaintext — by design; that is the point).
  - Step 3 ships only after step 2 is verified GREEN on a real DB copy, so the plaintext column is never dropped before its data is safely enveloped.

#### Scanners obtain the secret at runtime (decrypt-on-use)

- The scanner / filesystem client today reads `StorageRoot.Password` directly (`models/file.go:56`, consumed in `handlers/admin_handler.go`, `handlers/browse.go`, `main.go`). After migration it instead:
  1. loads the `StorageRoot` (now carrying `IdentityID`),
  2. looks up `share_identity` by `identity_id`,
  3. calls the secure store to **decrypt-on-use** (`Open(secret_ct, KEK)` from the §1 exported codec, or `securestorage.RetrieveCredentials` if the file-backed store is used) — returning the password ONLY into a short-lived local variable passed to the protocol client (SMB/FTP/etc.), never re-persisted, never logged.

- A thin identity.Service (new, in catalog-api services/) wraps the securestorage.Storage interface (securestorage.go:18) so call-sites depend on an interface (constructor injection per the project’s testing convention) and tests inject a fake store (unit-test-only fake, §11.4.27).

## Rollback / backup posture (§9.2)

- **Backup first, always:** before the destructive step 3 (column drop), take the §9.2 hardlinked .git/DB-file backup and a sqlite3 .backup / pg\_dump snapshot. The plaintext column is dropped only after step 2 GREEN proof + backup captured.
  - Steps 1–2 are non-destructive (additive table + NULL-out, with the down-migrations restoring structure). Step 3 is the only irreversible-for-plaintext step and is operator-gated per §11.4.122 / §9.2 sub-clause 6 (no automatic destructive run).
  - Roll-forward, never force: migration commits follow §11.4.113 (no force-push).
- 

## 4. Threat model

### 4.1 Where the master key lives

- **NOT in the database.** The DB holds only ciphertext (share\_identity.secret\_ct) + key\_id. The KEK lives outside the DB: env var HELIX\_IDENTITY\_MASTER\_KEY (preferred), OS keyring, or a 0600 key file (securestorage.go:248) as dev fallback. A DB dump alone yields no plaintext.
- Compromise model: an attacker needs **both** the DB (ciphertext) **and** the KEK source (env/keyring/keyfile). Separating them is the entire value over today’s password TEXT.

### 4.2 Blast radius

- **DB leak alone** (file copy, backup, SQL injection read): yields ciphertext only — no usable share passwords. Strict improvement over migrations/000001\_initial\_schema.sqlite.up.sql:12.
- **Host root / process memory:** a process holding the KEK can decrypt; this is inherent to any at-rest scheme and is bounded by the catalog-api “no sudo/root, user-level only” constraint (catalog-api/CLAUDE.md).
- **Key rotation:** key\_id/enc\_alg columns let one identity be re-enveloped under a new KEK without touching others — bounded rotation blast radius.



- **Per-identity isolation:** each share's secret is a separate row/ciphertext; cracking one AES-GCM blob does not reveal others (independent random nonces, `securestorage.go:341`).

### 4.3 What must NEVER be logged (§11.4.10)

- Never log: the KEK/DEK bytes; `secret_ct` decrypted plaintext; the `HELIX_IDENTITY_MASTER_KEY` value; any `password` / `api_token` / `ssh_key` material; the `.storage_key` file contents.
- Safe to log: `share_identity.id`, `name`, `scheme`, `key_id`, `enc_alg`, timestamps; the fact that a decrypt succeeded/failed (boolean), never the value.
- The decrypt-on-use path returns the secret into a local variable only; it is passed to the protocol client and must not be placed in struct fields that get JSON-serialized (note `StorageRoot.Password` had `json:"password"` at `models/file.go:56` — its removal also closes an API-response leak vector).
- The credential single-source convention (§11.4.10) means `.env / secrets/` stay git-ignored (`.env.example` carries placeholders only).

### 4.4 How tests avoid real secrets (§11.4.10 / §11.4.27)

- Tests use a fixed **test-only** 32-byte KEK from a test `env` var or a `t.TempDir()` key file — never a production key, never a real share password. `securestorage`'s own suite already follows this (`securestorage_test.go`, `_race_test.go` operate on temp dirs).
  - Unit tests may use a fake `securestorage.Storage` implementation (allowed only in unit tests, §11.4.27). Integration tests exercise the real `FileStorage/DB` path with synthetic credentials in a temp dir, asserting: (a) the DB column holds ciphertext  $\neq$  plaintext, (b) round-trip decrypt returns the synthetic value, (c) logs contain no plaintext (grep-the-captured-log assertion), (d) a DB dump grep finds no synthetic secret in the clear.
  - No real `.env` is read in tests; the §11.4.10 pre-store leak audit (`git ls-files | grep, git log -S`) runs before any operator key is committed to gitignored config.
- 

## 5. Recommended approach (summary)

Reuse `digital.vasic.security/pkg/securestorage` (already wired at `catalog-api/go.mod:43/72`) as the audited AES-256-GCM primitive, extending it **upstream** with an exported `Seal/Open` codec and a pluggable master-key source (`env/OS-keyring/KMS`) so secrets become a DB-column ciphertext rather than a sibling file. Store identities in a new

share\_identity table (scheme  $\in$  credentials/api\_token/ssh\_key, only secret\_ct enveloped under a KEK that lives outside the DB), and have StorageRoot reference an identity by identity\_id via share\_identity\_binding so the secret is stored exactly once and decrypted-on-use at scan time. Migrate in three reversible-until-the-last steps (create tables  $\rightarrow$  envelope existing storage\_roots.password values  $\rightarrow$  drop the plaintext column only after a §9.2 backup + GREEN proof), closing the §11.4.10 plaintext-password TEXT debt at models/file.go:56 / migrations/000001\_initial\_schema.sqlite.up.sql:12.